Introdução à Programação Orientada a Objetos

Classe, objeto, atributos, métodos, __init__ e self

Aula 18 • 30/10/2026 • Linguagem de Programação Python (80h) • CETAM
Profa. Priscila Gonçalves

Visão geral da aula

Data

Sexta-feira, 30/10/2026
Aula 18 • 4 tempos
Laboratório de informática

Habilidade técnica

Executar os conceitos iniciais de programação orientada a objetos (POO), com clareza e compreensão.

Projeto

Classe Usuario com cadastrar() e autenticar()

Habilidade técnica

- Executar os conceitos iniciais de POO, com clareza e compreensão
- Criar classes e objetos em Python
- Usar atributos, métodos, __init__ e self
- Modelar Cliente, Pacote e Usuario da agência

Objetos de conhecimento

- Introdução à Programação Orientada a Objetos
- Fundamentos de POO
- Classe, objeto, atributos e métodos
- __init__ e self

Roteiro dos 4 tempos

1º tempo

Aula expositiva: introdução e fundamentos de POO

2º tempo

Classe, objeto, atributos, métodos, __init__ e self (molde × pacote)

3º tempo

Atividade guiada: classes Cliente e Pacote

4º tempo

Desafio: classe Usuario com cadastrar() e autenticar()

1

1º tempo • Introdução à POO

Aula expositiva e dialogada

Retomada: onde estamos?

- Fase 5: do código solto à organização do sistema
- Aulas 15 e 16: funções, parâmetros e retorno
- Aula 17: arquivos para guardar os dados
- Hoje: uma nova forma de organizar o programa
- Programação Orientada a Objetos (POO)

Procedural × orientada a objetos

Procedural

O programa é uma sequência de passos e funções.
Dados e funções ficam separados.

Orientada a objetos

O programa é um conjunto de objetos.
Cada objeto reúne dados e ações.

Por que usar POO?

- Organiza programas grandes em partes pequenas
- Reúne dados e comportamento no mesmo lugar
- Facilita reaproveitar e manter o código
- Aproxima o programa do mundo real
- Python permite misturar funções e objetos

Dados e funções soltos

```
c1 = {"nome": "Ana", "email": "a@m.com"}

def apresentar(cliente):
    print(cliente["nome"], cliente["email"])

apresentar(c1)
```

- Dados e função separados
- Nada garante os campos
- A POO junta os dois

Objetos do mundo real

- Cliente: tem nome e e-mail; pode fazer reservas
- Pacote: tem destino, dias e preço; calcula o total
- Usuario: tem nome e senha; pode cadastrar e autenticar
- Todo objeto tem características e ações
- Características viram atributos; ações viram métodos

Vocabulário da aula

- Classe: molde que descreve um tipo de objeto
- Objeto (ou instância): exemplar criado a partir da classe
- Atributo: dado guardado no objeto
- Método: ação que o objeto sabe fazer
- Instanciar: criar um objeto a partir da classe

Fundamentos de POO

Abstração

Representar só o que importa do objeto

Encapsulamento

Guardar dados e ações juntos, protegendo o acesso

Herança

Reaproveitar uma classe em outra

Polimorfismo

Mesma ação, comportamentos diferentes

Da classe ao objeto



Uma classe pode gerar vários objetos, cada um com seus próprios dados.

2

2º tempo • Classe e objeto

Atributos, métodos, __init__ e self

Analogia: molde × pacote de viagem

Classe = molde

Define o que todo pacote tem (destino, dias, preço) e o que faz (calcular total).

Objeto = pacote

Cada pacote concreto criado a partir do molde: Parintins 3 dias, Tefé 4 dias.

Instanciar

Usar o molde para criar um pacote novo.

Classe e objeto

```
class Pacote:
    pass

p1 = Pacote()
p2 = Pacote()
print(type(p1))
```

- class cria o molde
- Pacote() cria um objeto
- p1 e p2 são objetos diferentes

__init__: o construtor

```
class Pacote:
    def __init__(self, destino, dias, preco_dia):
        self.destino = destino
        self.dias = dias
        self.preco_dia = preco_dia

p1 = Pacote("Parintins", 3, 280.0)
```

O que é o self?

- self é o próprio objeto que está sendo usado
- Sempre o primeiro parâmetro dos métodos
- self.destino: o atributo destino deste objeto
- Na chamada, o Python passa o self sozinho
- Sem self, o dado não fica guardado no objeto

Atributos

```
print(p1.destino)   # Parintins
print(p1.dias)      # 3

p1.dias = 4         # altera o atributo
print(p1.dias)      # 4
```

- Acesso com ponto: objeto.atributo
- Atributos podem ser alterados
- Cada objeto tem os seus

Métodos

```
class Pacote:
    def __init__(self, destino, dias, preco_dia):
        self.destino = destino
        self.dias = dias
        self.preco_dia = preco_dia

    def calcular_total(self):
        return self.dias * self.preco_dia

p1 = Pacote("Parintins", 3, 280.0)
print(p1.calcular_total())   # 840.0
```

Vários objetos, dados independentes

```
p1 = Pacote("Parintins", 3, 280.0)
p2 = Pacote("Tefé", 4, 150.0)

print(p1.calcular_total())   # 840.0
print(p2.calcular_total())   # 600.0

p2.dias = 5
print(p2.calcular_total())   # 750.0
```

- Mesmo molde, dados diferentes
- Alterar p2 não muda p1
- Método usa os dados do próprio objeto

Anatomia de uma classe

class

Palavra que cria a classe. Nome com inicial maiúscula.

__init__

Método especial que prepara o objeto.

Atributos

Dados do objeto, criados com self.nome.

Métodos

Funções da classe, com self como primeiro parâmetro.

Erros comuns

- Esquecer o self na definição do método
- Esquecer self. antes do atributo
- Esquecer os dois-pontos depois de class ou def
- Chamar o método sem parênteses: p1.calcular_total
- Confundir a classe (molde) com o objeto (pacote)

3

3º tempo • Atividade guiada

Classe Cliente e classe Pacote

Atividade avaliativa prática guiada

- Construiremos juntos as classes Cliente e Pacote
- Passo 1: classe Cliente com nome e e-mail
- Passo 2: classe Pacote com destino, dias e preço
- Passo 3: métodos e integração entre as classes
- Entrega: arquivo .py com testes

Passo 1: classe Cliente

```
class Cliente:
    def __init__(self, nome, email):
        self.nome = nome
        self.email = email

    def apresentar(self):
        print(f"Cliente: {self.nome} ({self.email})")
```

Usando a classe Cliente

```
c1 = Cliente("Ana", "ana@mail.com")
c2 = Cliente("Caio", "caio@mail.com")

c1.apresentar()
c2.apresentar()

# Cliente: Ana (ana@mail.com)
# Cliente: Caio (caio@mail.com)
```

- Dois objetos do mesmo molde
- Cada um com seus dados
- Teste com seus próprios dados

Passo 2: classe Pacote

```
class Pacote:
    def __init__(self, destino, dias, preco_dia):
        self.destino = destino
        self.dias = dias
        self.preco_dia = preco_dia

    def calcular_total(self):
        return self.dias * self.preco_dia

    def aplicar_desconto(self, taxa):
        return self.calcular_total() * (1 - taxa)
```

Testando o Pacote

```
p = Pacote("Tefé", 4, 150.0)
print(p.calcular_total())          # 600.0
print(p.aplicar_desconto(0.1))     # 540.0
print(p.aplicar_desconto(0.25))    # 450.0
```

- Um método chama outro
- self.calcular_total()
- Teste com outros valores

Passo 3: o Cliente faz reservas

```
class Cliente:
    # no __init__, acrescente:
    #     self.reservas = []

    def reservar(self, pacote):
        self.reservas.append(pacote)

    def total_reservas(self):
        total = 0
        for p in self.reservas:
            total += p.calcular_total()
        return total
```

Integrando os objetos

```
c = Cliente("Ana", "ana@mail.com")
p1 = Pacote("Parintins", 3, 280.0)
p2 = Pacote("Tefé", 4, 150.0)

c.reservar(p1)
c.reservar(p2)
print(c.total_reservas())   # 1440.0
```

- Objeto dentro de uma lista de outro
- O cliente soma o total
- 840.0 + 600.0 = 1440.0

Critérios da atividade

- Classes com __init__ e self corretos
- Atributos e métodos com nomes claros
- Objetos criados e testados com valores diferentes
- Integração entre Cliente e Pacote funcionando
- Código organizado e comentado

4

4º tempo • Desafio do projeto

Classe Usuario com cadastrar() e autenticar()

Desafio: classe Usuario

- Criar a classe Usuario com nome e senha
- Método cadastrar(): registra o usuário
- Método autenticar(senha): confere se a senha confere
- Testar com senha certa e errada
- Cuidado: senha em texto puro é só para estudo

Classe Usuario: cadastrar()

```
class Usuario:
    def __init__(self, nome, senha):
        self.nome = nome
        self.senha = senha
        self.cadastrado = False

    def cadastrar(self):
        self.cadastrado = True
        print(f"Usuário {self.nome} cadastrado!")
```

Método autenticar()

```
    def autenticar(self, senha):
        if not self.cadastrado:
            return False
        return self.senha == senha

u = Usuario("Ana", "abc123")
u.cadastrar()
print(u.autenticar("abc123"))   # True
print(u.autenticar("errada"))   # False
```

Cadastro e login com Usuario

```
nome = input("Nome: ")
senha = input("Senha: ")
u = Usuario(nome, senha)
u.cadastrar()

tentativa = input("Senha para entrar: ")
if u.autenticar(tentativa):
    print("Login realizado!")
else:
    print("Senha incorreta")
```

Dicas e cuidados

- Teste cada método separadamente
- autenticar devolve True ou False, e quem chama decide a mensagem
- Use dados fictícios e nunca senhas reais
- Em sistemas reais, senhas são guardadas com hash, nunca em texto puro
- Próxima aula: integrar tudo, com cadastro e login em arquivo

Fechamento

- Hoje: classe, objeto, atributos, métodos, __init__ e self
- Atividade: classes Cliente e Pacote
- Desafio: classe Usuario com cadastrar() e autenticar()
- Dúvidas? Obrigada!